Scalable Real-Time Notification System

Event-driven notifications, idempotency, rate limiting, retries, worker delivery

Project Type	Best For	Deliverables
Backend / Distributed Systems Demo	Amazon SDE I resume + interview explanation	Runnable code, demo script, tests, PDFs

1. Problem Statement

Applications need reliable notification delivery across email, SMS and push channels while handling provider failures and duplicate requests.

2. Architecture Flow

Client App -> Notification API -> Idempotency Layer -> Priority Queue -> Delivery Worker -> Provider Simulation -> Event Log

3. Core Features

- Create email, SMS and push notification jobs
- Idempotency keys prevent duplicate delivery
- Recipient rate limiting protects providers
- Priority queue ordering for important messages
- Retry handling for temporary provider failures
- Event log for delivery observability

4. API Surface

Endpoint	Purpose
POST /notifications	Queue a notification
POST /workers/process-next	Process next delivery job
GET /notifications	List queued and delivered notifications
GET /events	Inspect delivery events
POST /reset	Reset demo state

5. How to Run the Demo

Run these commands from the project folder:

```
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
pytest -q
```

6. Demo Talking Points

This demonstrates event-driven architecture. The API accepts work, the queue stores it, workers deliver it, and retries handle provider failure.

7. Resume Bullet

> Built an event-driven notification platform with idempotent APIs, priority queues, retry handling, recipient rate limiting, delivery workers, and event tracking.

8. What to Say in Interview

- Why I built it: to practice backend workflows that need correctness, reliability and scalability.
- Main challenge: keeping state transitions correct when retries or failures happen.
- What I learned: data structures, APIs, testing, failure handling, and clean modular design.
- Future scale-up: replace in-memory components with Redis/Kafka/SQS, add auth, metrics, tracing and cloud deployment.

9. Demo Output Snapshot

POST /notifications


```
{
"deduplicated": false,
"notification": {
"id": "864f61e5",
"recipient": "u@example.com",
"channel": "email",
"message": "fail once then deliver",
"priority": 9,
"attempts": 0,
"status": "QUEUED"
}
}
```


POST /workers/process-next


```
{
"id": "864f61e5",
"recipient": "u@example.com",
"channel": "email",
"message": "fail once then deliver",
"priority": 9,
"attempts": 1,
"status": "RETRY",
"result": "temporary provider failure"
}
```


POST /workers/process-next again

```
{
"id": "864f61e5",
"recipient": "u@example.com",
"channel": "email",
"message": "fail once then deliver",
"priority": 9,
"attempts": 2,
"status": "DELIVERED",
"result": "sent via email"
}
```